Лабораторная работа №1
Использование потоков

Цель работы

Целью работы является ознакомление с основными принципами многопоточности в языке Java, возможностью написания программ с использованием потоков.

1. Теоретический материал

Понятие потока

В Java под термином «поток» подразумевают два разных понятия:

– экземпляр класса `java.lang.Thread`;
– поток выполнения программы.

Экземпляр `Thread` – это обычный объект, который как и другие объекты в Java имеет переменные и методы.

Поток выполнения (thread of execution) – это отдельный процесс, имеющий свой собственный стек вызовов. В Java для каждого потока существует свой стек вызовов. Например, метод `main()` выполняется в потоке, который называется главным (main thread). Если посмотреть на стек вызовов, то можно увидеть, что метод `main()` стоит первым в стеке, т.е. находится в самом низу.

Для просмотра стека выполним упражнение 1, в котором из метода `main` вызывается метод, в котором в свою очередь вызывается еще один метод.

Упражнение 1. Стек главного потока

```
package lab1Thread;
public class t01 {
    public static void main(String[] args) {
        fmeth();
        System.out.println("Поток main");
    }
    private static void fmeth() {
        System.out.println("Поток fmeth");
        smeth();
    }
    private static void smeth() {
        System.out.println("Поток smeth");
    }
}
```

Поставим точку останова в самом внутреннем методе и запустим программу в режиме `debug`. Для просмотра стека потока надо подключить вид *Debug* (через меню Window – Show View – Debug). Из содержимого стека

видно, последовательность вызова методов (`main – fmeth – smeth`).

```
2
3  public class t01 {
4      public static void main(String[] args) {
5          fmeth();
6          System.out.println("Поток main");
7      }
8      private static void fmeth() {
9          System.out.println("Поток fmeth");
10         smeth();
11     }
12     private static void smeth() {
13         System.out.println("Поток smeth");
14     }
15 }
```

```
Console  Tasks  Debug
t01 [Java Application]
  lab1Thread.t01 at localhost:2804
    Thread [main] (Suspended (breakpoint at line 16 in t01))
      t01.smeth() line: 16
      t01.fmeth() line: 12
      t01.main(String[]) line: 6
  E:\Program Files\Java\jre7\bin\javaw.exe (01 июля 2017 г., 23:15:55)
```

Java-потоки реализуются через потоки операционной системы, однако последовательность выполнения потоков определяется планировщиком потоков JVM, а не планировщиком операционной системы.

Создание потока

Определить поток можно двумя способами:

1. Расширить класс `java.lang.Thread`;
2. В классе реализовать интерфейс `Runnable`.

Ввиду отсутствия множественного наследования более предпочтительным способом является реализация интерфейса `Runnable`. Реализация интерфейса `Runnable` дает возможность расширить любой другой класс и в то же время определить поведение, которое будет выполняться в отдельном потоке. Действия, которые должен выполнить поток, помещаются в методе `run()`.

```
class TaskRunnable implements Runnable {
    @Override
    public void run() {
        System.out.println(
            "Работа выполняется в TaskRunnable");
    }
}
```

Инициализация потока

После создания класса, реализующего `Runnable`, для его запуска все равно целесообразно создать экземпляр класса `Thread`, а уже ему назначить объект созданного класса.

```
TaskRunnable r = new TaskRunnable();
Thread t = new Thread(r);
```

Это делается для разделения логики на два класса – класса `Thread`, реализующего логику потока, и класса реализующего `Runnable` для описания работы, которая должна выполняться в отдельном потоке. Другими словами: класс `Thread` – исполнитель, а класс `Runnable` – это выполняемая работа.

Один экземпляр `Runnable` можно передать нескольким объектам `Thread`:

```
TaskRunnable tr = new TaskRunnable();
Thread t1 = new Thread(tr);
Thread t2 = new Thread(tr);
```

Запуск потока

Созданный поток запускается методом `start()`:

```
tr.start();
```

При старте потока происходит следующее:

- стартует новый поток выполнения (с новым стеком вызовов);
- поток переходит из состояния new (новый) в состояние работоспособный (runnable);
- когда поток получает шанс выполниться, он вызывает метод `run()`.

При этом начало выполнения потока определяется планировщиком JVM.

Запуск и выполнение нескольких потоков

Как уже отмечалось, запуском потоков управляет планировщик JVM. Если используется однопроцессорный ПК, то потоки будут выполняться последовательно, причем нельзя заранее определить последовательность их выполнения.

Рассмотрим пример запуска нескольких потоков. Для отслеживания, какой поток выполняется в данный момент времени, потокам присвоим имена методом `setName()`.

Упражнение 2. Запуск нескольких потоков.

```
public class ManyThread {
    public static void main(String[] args) {
        NameThread nr = new NameThread ();
```

```
        Thread t1 = new Thread(nr);
        Thread t2 = new Thread(nr);
        t1.setName("Первый поток");
        t2.setName("Второй поток");
        t1.start();
        t2.start();
    }
}
class NameThread implements Runnable {
    public void run() {
      int s = 0;
      for (int x = 1; x <= 3; x++) {
         s+=x;
        System.out.println("Запущен "
               + Thread.currentThread().getName()
               + ", s = " + s);
      }
    }
}
```

В потоке выполняется суммирование натуральных чисел от 1 до 3. Класс `NameThread` реализует интерфейс `Runnable`. В методе `main` создаются 2 потока `t1`, `t2`. В конструктор этих потоков передается объект класса `NameThread`. Потоки запускаются методом `start` и выполняют действия, записанные в методе `NameThread.run`, т.е. суммируют натуральные числа. Причем каждый поток выполняет эту задачу независимо от другого.

Результат увидим в консоли (последовательность потоков при повторном запуске может быть другой).

```
Search  Debug  Shell  Servers  Console
<terminated> t02 [Java Application] E:\Program Files\Java\jr
Запущен Первый поток, s = 1
Запущен Второй поток, s = 1
Запущен Первый поток, s = 3
Запущен Первый поток, s = 6
Запущен Второй поток, s = 3
Запущен Второй поток, s = 6
```

Планировщики потоков

Планировщик, являющийся частью виртуальной машины Java, управляет последовательностью запуска нескольких работоспособных (готовых к выполнению) потоков, т.е. находящихся в состоянии runnable.

Планировщики бывают двух видов: с приоритетом и без приоритета.

Планировщик с приоритетом предоставляет определённый отрезок времени поочередно всем потокам, которые запущены в системе.

Планировщик решает, какой поток возобновит работу (или начнет работу) следующим через некоторый постоянный период времени. После запуска через определённый промежуток времени выполняющийся поток будет приостановлен и следующий поток возобновит свою работу.

Планировщик без приоритета запускает определенный поток, и он будет выполняться без прерывания до тех пор, пока не закончит свою работу.

Как правило, JVM планировщик осуществляет планирование на основе приоритетов. Это хорошо видно из результатов выполнения программы упражнения 2. Причем потоки не строго чередуются один за другим.

Когда метод `run()` потока завершится, поток перестает быть потоком исполнения, стек вызовов удаляется, а поток считается мертвым (`dead`). С этого момента поток – это обычный объект `Thread`. Поток никогда не может быть запущен повторно. При попытке повторного запуска выполненного потока методом `start()` будет выдано исключение `IllegalThreadStateException`. Запустить поток можно только из состояния `new` (новый), а состояние `new` он имеет только перед первым запуском.

Жизненный цикл потока выполнения

Поток может находиться в одном из следующих состояний:

1. **`new` (новый)**. После создания экземпляра потока, он находится в состоянии **`new`** до тех пор, пока не будет вызван метод `start()`. В этом состоянии поток еще не готов к выполнению.
2. **`runnable` (готовый к выполнению)**. Поток переходит в состояние готовности к выполнению сразу после вызова метод `start()`. В это состояние поток также может перейти из состояний работы или блокировки.
3. **`running` (работающий)**. Готовый к выполнению поток переводится в состояние работы планировщиком потоков из перечня потоков `runnable`.
4. **`waiting` (ожидающий) / `blocked` (заблокированный) / `sleeping` (спящий)**. Эти состояния характеризуют поток, приостановленный по одной из причин. В этом состоянии. поток не работает, но он может вернуться в состояние готовности к работе определенным способом. Рассмотрим примеры приостановки работающего потока и возврата его состояние готовности. Поток может быть заблокирован, если занят требуемый ему ресурс. Поток будет разблокирован после освобождения нужного ресурса. Поток может быть переведен в состояние засыпания методом `sleep()` на заданный интервал времени. Поток просыпается по истечении заданного интервала времени. Поток переводится в состояние ожидания методом `wait()`. Вывод потока из состояния ожидания

выполняется вызовом метода `notify()` для конкретного потока или `notifyAll()` для всех ожидающих потоков. Следует учитывать, что приостановить поток может только себя в своем методе `run()`, но приостановить другие потоки он не может, так как указанные выше методы являются статическими и не применимы к экземплярам любых объектов.

5. **`dead` (мертвый)**. Поток становится мертвым после завершения выполнения метода `run()`. Мертвый поток нельзя перевести в предыдущие состояния.

Переход между состояниями можно представить схемой, приведенной на рис.1.

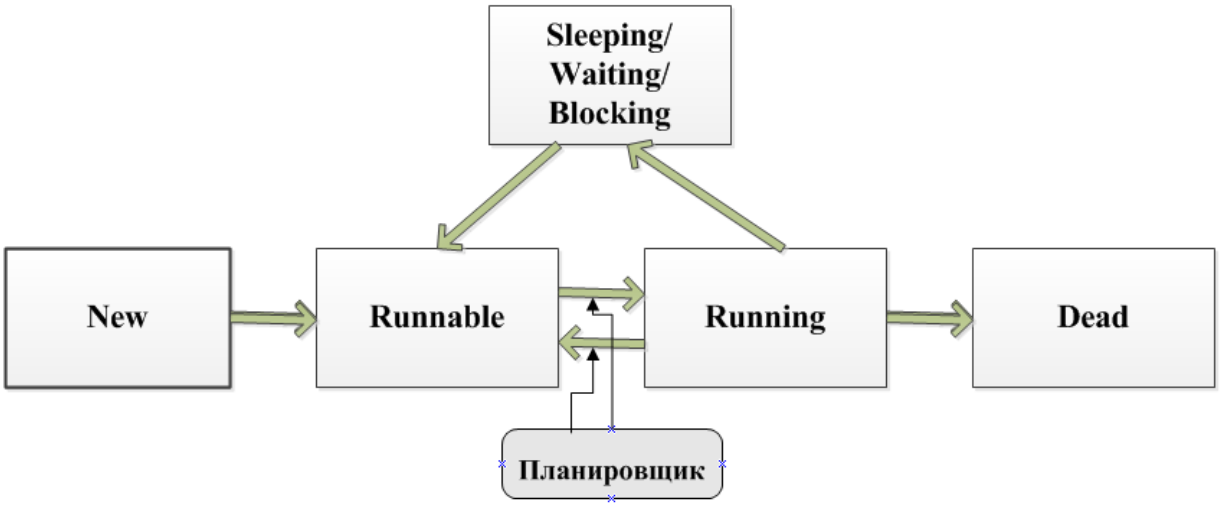


Рис.1 Жизненный цикл потока

Приостановка выполнения потока

Существует несколько способов приостановить выполнение работающего потока.

1. **Метод `sleep(время)`** прекращает выполнение текущего потока на заданный промежуток времени.
2. **Метод `yield()`** делает попытку перевести текущий поток из состояния `running` в состояние `runnable`, чтобы дать поработать другому потоку с таким же приоритетом.
3. **Метод `join()`** гарантированно останавливает выполнение текущего потока до завершения выполнения потока, к которому он присоединен (как правило, если зависит от результатов его выполнения).
4. Метод **`wait()`** приостанавливает поток на неопределенное время до момента вызова метода `notify()` или `notifyAll()`.
5. Поток приостанавливается также в том случае, если не может получить блокировку на объект, метод которого пытается выполнить.

Рассмотрим эти способы подробнее.

Метод `sleep()` – это статический метод класса `Thread`. Метод вызывает засыпание текущего потока на заданное время (в миллисекундах). Переключение потока в состояние сна может выполняться для притормаживания его выполнения или для того, чтобы дать возможность нескольким потокам выполняться по очереди.

Пример

```
try {
  Thread.sleep(3*60*1000); // Сон на 3 минуты
} catch (InterruptedException ex) { }
```

Пробуждение потока по истечении заданного промежутка времени еще не означает, что он сразу же перейдет в состояние `running`. Поток просто переходит в состояние `runnable`, а значит становится доступным для планировщика потоков. Поэтому, время, переданное в метод `sleep()`, это минимальное время, в течение которого поток будет приостановлен.

Еще раз отметим тот факт, что метод `sleep()` является статическим и потому может перевести в сон только текущий поток.

Упражнение 3. Изменим метод `run` потока из Упражнения 2, переводя текущий поток в состояние сна на 1 сек.

```
public void run() {
   int s = 0;
   for (int x = 1; x <= 3; x++) {
      s+=x;
      System.out.println("Запущен "+
           Thread.currentThread().getName()
           + ", s = " + s);
      try {
         Thread.sleep(1000);
      }
      catch (InterruptedException ex) {
         ex.printStackTrace();
      }
   }
}
```

Результат выполнения

```
Search  Debug  Shell  Servers  Console
<terminated> t02 [Java Application] E:\Program Files\Java\j
Запущен Первый поток, s = 1
Запущен Второй поток, s = 1
Запущен Первый поток, s = 3
Запущен Второй поток, s = 3
Запущен Второй поток, s = 6
Запущен Первый поток, s = 6
```

Видно, что потоки выполняются поочередно, хотя это не гарантированно.

Таким образом, метод `sleep()` является лучшим способом предоставить возможность выполниться всем потокам. Или, по крайней мере, можно гарантировать, что один поток не будет выполняться все время до завершения своего метода `run()`.

Метод `join()` является нестатическим методом класса `Thread` и позволяет присоединить текущий поток к другому потоку. Рассмотрим следующий пример. Поток B на определенном этапе своего выполнения использует результат, полученный при выполнении другого потока A, а потому он может продолжить свою работу только после завершения потока A. Поэтому нужно присоединить поток B к потоку A. В этом случае поток B переходит в состояние блокировки и будет находится в данном состоянии до завершения потока A. Такая блокировка иллюстрируется следующим кодом в методе `run()` потока B:

```
Thread tA = new Thread();
tA.start();
tA.join();
```

В приведенном создается экземпляр tA потока A, который стартует, а затем текущий поток (в который вставлен данный код) приостанавливается (блокируется), становясь в хвост вслед за потоком A. Это блокировка продолжается до момента завершения потока A, т.е до завершения метода `run()` потока A.

Работа **метода `yield()`** базируется на понятии приоритетов потоков. Любой поток запускаются с некоторым приоритетом, который задается числом от 1 (минимальный) до 10 (максимальный). По умолчанию при создании потока ему присваивается средний приоритет равный 5. Изменить приоритет потока можно методом `setPriority(приоритет)`. В следующем примере потоку присваивается приоритет 9:

```
Thread t = new Thread(r);
t.setPriority(9);
```

Планировщик потоков, как правило, осуществляет планирование на основе приоритетов. Это означает, что если запускается поток с приоритетом выше, чем у текущего, то планировщик попытается запустить этот поток, приостановив текущий, который он переведет в состояние `runnable`. Однако абсолютно гарантировать данное поведение нельзя, т.к. в очереди на выполнение могут стоять потоки с более высоким приоритетом.

Метод `yield()` как раз и используется для того, чтобы заставить планировщик перейти к выполнению другого потока, стоящего в очереди на выполнение. При этом метод `yield()` не пытается приостановить поток, переводя его в состояние `waiting/sleeping/blocking`, а пытается перевести его из состояния `running` в `runnable`. И как уже упоминалось ранее вызов этого метода не всегда гарантирует ожидаемый результат.

Метод `wait()` будет рассмотрен позже при описании взаимодействия потоков. Блокировки рассматриваются в следующем разделе.

Работа потоков с одним ресурсом. Синхронизация

На практике часто встречается ситуация, когда два разных потока имеют доступ к одному объекту (общему ресурсу), и оба эти потока будут вызывать методы, которые изменяют состояние объекта.

Простым примером является работа с банковским счетом, на котором лежит некоторая сумма. Допустим, что счет может быть использован только для одной операции снятия денежных средств. Чтобы счет не ушел в отрицательную область, необходимо перед списанием проверить наличие на счете требуемой для снятия суммы.

Поэтому операция списания средств со счета выполняется в два этапа:

- проверка баланса на счете;
- если на счету достаточно средств, то производится списание.

Если со счетом работает один пользователь, то проблем не будет. Проблемы возникнут, если со счетом начинают работать два и более пользователя.

Это можно увидеть из программы упражнения 4.

Упражнение 4. 2 пользователя снимают средства с банковского счета. Каждый пользователь выполняет 3 операции списания, что определяется циклом. Для простоты списание выполняется на одну и ту же сумму. Роль потока в данном примере выполняет пользователь (класс `User`), снимающий средства, а объектом или ресурсом, с которым взаимодействуют потоки, выступает банковский счет (класс `Schet`).

Класс Банковский счет (`Schet`)

```
public class Schet {
    //  Баланс на счете
    private int balance;
    //  Конструктор (определяется начальный баланс)
    public Schet(int startBalance){
       balance=startBalance;
    }
    // Метод определения баланса
    public int getBalance() {
        return balance;
    }
    //  Метод списания средств
    public void spisanie(int sum) {
        balance = balance - sum;
    }
}
```

Класс Пользователь (User)

```
public class User implements Runnable {
   private Schet schet;
   // Конструктор
   public User (Schet sch) {
       schet=sch;
   }
   // Выполняемая работа (3 операции снятия средств)
   @Override
   public void run() {
     for (int i = 0; i < 3; i++) {
         makeSpis(10);
         try {
            Thread.sleep(500);
         } catch (InterruptedException ex) {
         }
     }
   }
   //  Метод списания
   private void makeSpis(int s) {
     int balance=schet.getBalance();
     System.out.println(
         Thread.currentThread().getName()
         + " совершил проверку счета"+
         " Остаток="+balance);
     if (balance>= s) {
        schet.spisanie(s);
        System.out.println(
            Thread.currentThread().getName()
            + " вывел средства=" +s
            + " Остаток="+ schet.getBalance());
     }else {
        System.out.println("Недостаточно средств "+
               "на счете для клиента с именем "
              + Thread.currentThread().getName()
              + ". Доступно для вывода: " +
              schet.getBalance());
     }
   }
}
```

Класс тестирования

```
public class TestUserSchet {
    public static void main(String[] args) {
      Schet sch = new Schet(20);
```

```
        User u = new User(sch);
        Thread t1 = new Thread(u);
        Thread t2 = new Thread(u);
        t1.setName("Пользователь 1");
        t2.setName("Пользователь 2");
        t1.start();
        t2.start();
    }
}
```

После нескольких запусков программы (тестового класса) были получены различные результаты: нормального выполнения и некорректного выполнения снятия денежных средств со счета.

а) нормальное выполнение

```
Problems  @ Javadoc  Declaration  Search  Console  Debug
<terminated> TestUser [Java Application] E:\Program Files\Java\jre1.8.0_161\bin\javaw.exe (5 мар. 2019 г., 11:44:07)
Пользователь 1 совершил проверку счета Остаток=30
Пользователь 1 вывел средства=10 Остаток=20
Пользователь 2 совершил проверку счета Остаток=30
Пользователь 2 вывел средства=10 Остаток=10
Пользователь 1 совершил проверку счета Остаток=10
Пользователь 1 вывел средства=10 Остаток=0
Пользователь 2 совершил проверку счета Остаток=0
Недостаточно средств на счете для клиента с именем Пользователь 2. Доступно для вывода: 0
Пользователь 1 совершил проверку счета Остаток=0
Недостаточно средств на счете для клиента с именем Пользователь 1. Доступно для вывода: 0
Пользователь 2 совершил проверку счета Остаток=0
Недостаточно средств на счете для клиента с именем Пользователь 2. Доступно для вывода: 0
```

б) некорректное выполнение

```
Problems  @ Javadoc  Declaration  Search  Console  Debug
<terminated> TestUser [Java Application] E:\Program Files\Java\jre1.8.0_161\bin\javaw.exe (5 мар. 2019 г., 11:45:08)
Пользователь 2 совершил проверку счета Остаток=30
Пользователь 2 вывел средства=10 Остаток=20
Пользователь 1 совершил проверку счета Остаток=30
Пользователь 1 вывел средства=10 Остаток=10
Пользователь 1 совершил проверку счета Остаток=10
Пользователь 1 вывел средства=10 Остаток=0
Пользователь 2 совершил проверку счета Остаток=10
Пользователь 2 вывел средства=10 Остаток=-10
Пользователь 1 совершил проверку счета Остаток=-10
Недостаточно средств на счете для клиента с именем Пользователь 1. Доступно для вывода: -10
Пользователь 2 совершил проверку счета Остаток=-10
Недостаточно средств на счете для клиента с именем Пользователь 2. Доступно для вывода: -10
```

Анализ результатов последнего сеанса показывает, что пользователи вывели больше средств (6 раз по 10 единиц), чем была первоначальная сумма 30 единиц.

Исключить некорректное выполнение списания средств не сложно Необходимо гарантировать, что два шага (проверка баланса и списание средств) будут выполняться, как одна транзакция, т.е. отдельные шаги не будут разделяться вклиниванием другого потока. Такая операция называется «*атомарной*». На время выполнения атомарной операции необходимо накладывать блокировку на разделяемый ресурс (в примере это банковский счет). В таком случае даже, если поток, который начал операцию первым,

заснет после проверки баланса, то другой не сможет начать операцию до конца выполнения операции первым потоком. Т.е второй поток будет ждать, когда первый поток проснется и завершит операцию снятия средств со счета.

Для установки такой блокировки необходимо синхронизировать метод, выполняющий операцию списания средств.

Упражнение 5. Синхронизированная версия упражнения 4. Изменим программу упражнения 4, добавив ключевое слово `synchronized` к методу `makeSpis()`:

```
private synchronized void makeSpis(int s) {...}
```

Результат

```
Problems  @ Javadoc  Declaration  Search  Console  Debug
<terminated> TestUserSync [Java Application] E:\Program Files\Java\jre1.8.0_161\bin\javaw.exe (5 мар. 2019 г., 11:33:57)
Пользователь 1 совершил проверку счета Остаток=30
Пользователь 1 вывел средства=10 Остаток=20
Пользователь 2 совершил проверку счета Остаток=20
Пользователь 2 вывел средства=10 Остаток=10
Пользователь 2 совершил проверку счета Остаток=10
Пользователь 2 вывел средства=10 Остаток=0
Пользователь 1 совершил проверку счета Остаток=0
Недостаточно средств на счете для клиента с именем Пользователь 1. Доступно для вывода: 0
Пользователь 1 совершил проверку счета Остаток=0
Недостаточно средств на счете для клиента с именем Пользователь 1. Доступно для вывода: 0
Пользователь 2 совершил проверку счета Остаток=0
Недостаточно средств на счете для клиента с именем Пользователь 2. Доступно для вывода: 0
```

Анализ результатов показывает, что снятие происходит только при наличии положительного баланса.

Синхронизация происходит за счет блокировок. Каждый объект в Java имеет встроенную блокировку, которая начинает действовать только при выполнении синхронизированных методов или синхронизированных участков кода. При входе в синхронизированный нестатический метод автоматически устанавливается блокировка на используемый ресурс (разделяемый объект), закрепленная за текущим экземпляром класса потока. Текущим является экземпляр потока, код которого выполняется в данный момент.

Поскольку возможна только одна блокировка на объект, то при установке блокировки одним потоком, ни какой другой поток не может ее захватить, пока первый не снимет блокировку по завершении операции с эти объектом. Это означает, что другой поток не сможет войти в синхронизированный код пока блокировка не будет снята с объекта (общего ресурса).

Перечислим ключевые моменты блокировки и синхронизации:

- синхронизированными могут быть только методы или участки кода метода;
- каждый объект (ресурс) может иметь только одну блокировку.

- только один поток сможет выполнить синхронизированный метод. Другой поток будет ждать, пока первый не закончит выполнение метода.
- синхронизированными следует делать методы, выполняющие действия с разделяемым (общим) ресурсом;
- если класс содержит не синхронизированные методы, то несколько потоков могут получить доступ к таким методам класса;
- если поток переводится в сон, он удерживает все свои блокировки и на период засыпания;.
- поток может захватить несколько блокировок, если они относятся к разным объектам;
- Если поток пытается войти в синхронизированный метод, а блокировка уже захвачена другим потоком, то поток переходит в некий пул для этого объекта и находится там пока блокировка не снимется. Когда блокировка освободится, этот поток снова перейдет в состояние `runnable`.

В следующем примере показана блокировка (синхронизация) не на весь метод, а лишь фрагмент его кода, связанный с работой с разделяемым ресурсом. Ключевое слово синхронизации (блокировки) применяется к объекту ресурса (банковского счета):

```
    public void run() {
            for (int i = 0; i < 3; i++) {
             synchronized (sch){
                makeSpis(10);
             }
            }
    }
```

Упражнение 6. Усложним задачу, создав два различных потока: один будет добавлять средства, а другой списывать средства.

```
//  Тестовый класс, запускающий потоки
public class BankSchet {
    public static void main(String[] args) {
        Schet1 sch = new Schet1(30);
        Tin ti = new Tin(sch);
        Tout to = new Tout(sch);

        Thread t1 = new Thread(ti);
        Thread t2 = new Thread(to);

        t1.setName("Пользователь in");
        t2.setName("Пользователь out");

        t2.start();
        t1.start();
```

```
    }
}
//  Класс счета
class Schet1 {
    private int balance;
    //  Конструктор
    public Schet1(int balance) {
      this.balance=balance;
    }
    //  Метод проверки баланса
    public int getBalance() {
        return balance;
    }
    //  Метод списания средств
    public void spisanie(int sum) {
        balance = balance - sum;
    }
    //  Метод зачисления средств
    public void zachislenie(int sum) {
        balance = balance + sum;
    }
}
//  Класс зачисления средств
class Tin implements Runnable {
    private Schet1 sch;
    //  Конструктор класса
    public Tin (Schet1 sch){
     this.sch=sch;
    }
    @Override
    public void run() {
        //  Выполняет 3 операции зачисления
        for (int i = 0; i < 3; i++) {
           synchronized (sch){
               makeZach(10);
           }
           try {
             Thread.sleep(1500);
           } catch (InterruptedException e) {
             e.printStackTrace();
           }
        }
        System.out.println(Thread.currentThread().getName()+
           " завершил работу");
    }
```

```
    //  Метод зачисления средств
    private  void makeZach(int s) {
        sch.zachislenie(s);
        System.out.println(Thread.currentThread().getName()
                    +" зачислил средства=" +s
                    +" Остаток="+sch.getBalance());
    }
}
//  Класс вывода средств
class Tout implements Runnable {
    private Schet1 sch;
    //  Конструктор класса
    public Tout (Schet1 sch){
     this.sch=sch;
    }
    @Override
    public void run() {
        //  Выполняет 5 операций вывода средств
        for (int i = 0; i < 5; i++) {
          synchronized (sch){
               makeSpis(10,i);
          }
          try {
             Thread.sleep(500);
          } catch (InterruptedException ex) {
          }
        }
        System.out.println(Thread.currentThread().getName()+
           " завершил работу");
    }
    //  Метод списания средств
    private void makeSpis(int s, int i) {
      //  Определение баланса
      int ost=sch.getBalance();
      //  Проверка необходимого остатка
      if ( ost>= s) {
         System.out.println(Thread.currentThread().getName()
               + " совершил проверку счета. Остаток = "+ost);
         sch.spisanie(s);
         System.out.println(Thread.currentThread().getName()
          + " вывел средства="+s+" Остаток="+sch.getBalance());
      } else {
        System.out.println("Недостаточно средств на счете для"
         +" клиента с именем "+Thread.currentThread().getName()
          +  " Доступно для вывода: " + ost);
```

```
            }
        }
}
```

Результат

```
Search  Debug  Shell  Servers  Console
<terminated> T05_1 [Java Application] E:\Program Files\Java\jre7\bin\javaw.exe (05 июля 20
Пользователь out совершил проверку счета. Остаток = 30
Пользователь out  вывел средства=10 Остаток=20
Пользователь in зачислил средства=10 Остаток=30
Пользователь out совершил проверку счета. Остаток = 30
Пользователь out  вывел средства=10 Остаток=20
Пользователь out совершил проверку счета. Остаток = 20
Пользователь out  вывел средства=10 Остаток=10
Пользователь in зачислил средства=10 Остаток=20
Пользователь out совершил проверку счета. Остаток = 20
Пользователь out  вывел средства=10 Остаток=10
Пользователь out совершил проверку счета. Остаток = 10
Пользователь out  вывел средства=10 Остаток=0
Пользователь out завершил работу
Пользователь in зачислил средства=10 Остаток=10
Пользователь in завершил работу
```

Взаимодействие потоков

Рассмотрим каким образом потоки могут взаимодействовать друг с другом. В классе `Object` есть три метода `wait()`, `notify()`, и `notifyAll()`, которые позволяют потоку сообщать информацию о своем состоянии другим, заинтересованным в этой информации потокам.

Например. Есть два потока. Первый занимается доставкой почты, второй – обрабатывает почту. Обработчик почты должен постоянно проверять, есть ли в очереди письма, которые требуется обработать. Используя механизмы ожидания (`wait`) и уведомления (`notify`), поток-обработчик почты может посмотреть состояние очереди и ,если там ничего нет, то он остается в состоянии ожидания. Другими словами, метод `wait()` отправляет поток в состояние ожидания, из которого его выводит поток, отправляющий уведомление методом `notify()`. При этом метод `notify()` отправляет уведомление конкретному потоку, а `notifyAll()`- всем заинтересованным потокам.

Важный момент при использовании механизма `wait/notify` заключается в следующем: методы `wait()`, `notify()`, и `notifyAll()` должны вызываться из синхронизированного (`synchronized`) контекста, иначе можно получить исключительную ситуацию.

Рассмотрим пример из двух потоков, которые зависят друг от друга, и покажем, как использовать `wait()` и `notify()` чтобы задействовать потоки в нужный момент.

Методы `wait()` и `notify()` являются методами класса `Object`. Каждый объект имеет блокировку. Таким же образом, каждый объект может иметь список потоков, которые ждут от него сигнала (уведомления). Поток попадает в этот список ожидающих при помощи вызова `wait()` целевого объекта. С этого момента он не выполняет ни каких действий пока не будет

вызван метод `notify()`. Если один объект ждут несколько потоков, то после вызова `notify()`, планировщиком потоков будет выбран один поток (какой именно не известно), который приступит к исполнению. Если ожидающих потоков нет, то ничего и не произойдет.

Упражнение 7. Рассмотрим код, в котором один объект ждет уведомления от другого.

```
public class ThreadWithWait {
  public static void main(String[] args) {
    ThreadB tB = new ThreadB();
    tB.start();
    synchronized (tB) {
       try {
         System.out.println(
           "Ждем пока поток b выполнится...");
         tB.wait();
       } catch (InterruptedException e) {
       }
       System.out.println("Сумма = " + tB.total);
    }
  }
}

class ThreadB extends Thread {
   int total=0;
   public void run() {
     synchronized (this) {
      // В цикле суммируем 10 натуральных чисел
      // после каждой операции задержка 0,5с
       for (int i = 0; i < 10; i++) {
         try {
             Thread.sleep(500);
        } catch (InterruptedException e) {
             e.printStackTrace();
        }
        total += i;
       }
       System.out.println(
           "Возврат к выполнению основного потока...");
       notify();
     }
   }
}
```

Эта программа содержит два потока: `ThreadWithWait` – основной поток и `ThreadB` – поток, который вычисляет сумму всех чисел от 0 до 9.

После того, как выполнится строка `tB.start()`, `ThreadWithWait` продолжит свое выполнение и может попасть в строку `System.out.println(...)` до того, как `ThreadB` закончит свои расчеты. Чтобы предотвратить такое развитие событий, добавлен метод `tB.wait()`, приостанавливающий его выполнение до завершения работы потока `ThreadB`.

Обратите внимание на строку `synchronized(tB)`. Она необходима для того, чтобы `ThreadWithWait` получил блокировку на `tB`. Чтобы вызвать методы `wait()` или `notify()`, поток должен иметь блокировку объекта, на котором вызываются эти методы. Если вызвать метод `wait()` из потока, не имеющего блокировку на объект, на котором вызывается `wait()`, то возникнет исключение `IllegalMonitorStateException`. Так как ожидающий поток может быть прерван таким же образом, как и спящий (`sleeping`), то блок кода, в котором вызывается `wait()` заключен в конструкцию `try/catch`, что позволяет перехватить исключение `InterruptedException`.

Использование ***`notifyAll()`***

В большинстве случаев необходимо уведомить не один поток, ждущий конкретный объект, а все потоки. Для этого используется метод `notifyAll()`, который выводит все потоки из состояния ожидания и переводит их в состояние `runnable`. Это актуально если есть несколько потоков, ожидающих один объект и необходимо гарантировать, что все потоки получат соответствующие уведомления. При вызове метода `notifyAll()` все потоки получат уведомление и начнут конкурировать за право получить блокировку.

Упражнение 8. Рассмотрим пример, в котором один поток выполняет вычисления, а множество потоков-читателей ждут результата расчетов.

Основной класс

```
public class Reader extends Thread {
    Calculator c;
    //  Конструктор класса
    public Reader(Calculator calc) {
        c = calc;
    }
    // Выполняемая в потоке работа
    public void run() {
        synchronized (c) {
            try {
                System.out.println("Вычисление...");
                c.sleep(100);
                c.wait();
            } catch (InterruptedException e) {
```

```
            }
            System.out.println("Total равно: " + c.total);
        }
    }
    public static void main(String[] args) {
        Calculator calculator = new Calculator();
        new Reader(calculator).start();
        new Reader(calculator).start();
        new Reader(calculator).start();
        calculator.start();
    }
}
```

Класс Калькулятор (Calculator)

```
class Calculator extends Thread {
    int total;

    public void run() {
        synchronized (this) {
            for (int i = 0; i < 100; i++) {
                total += i;
            }
            notifyAll();
        }
    }
}
```

Программа запускает три потока, которые ожидают получения результатов расчета, а затем запускает поток, который производит расчеты. Следует учесть, что если в методе `run()` из класса `Calculator` вызвать `notify()` вместо `notifyAll()`, то будет уведомлён только один поток.

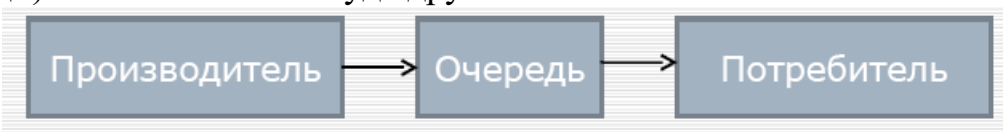
Использование **`wait()`** *в цикле*

Пример взаимодействующих классов читатель/калькулятор имеет одну общую проблему. Есть один поток, который вызывает `wait()` и другие потоки, которые вызывают `notify()` или `notifyAll()`. Но что произойдет если, например, поток `Calculator` запустится первым и вызовет `notify()` до того, как потоки-читатели начнут ожидание? К сожалению, когда читатели запускаются, они сразу же уходят в состояние ожидания. Они не делают ничего, чтобы убедиться, что событие, которого они ждут, уже произошло. Так, если Calculator уже вызвал notifyAll(), он больше не будет делать этого и читатели будут ждать вечно. Очевидно, когда нужно ждать какое-то событие, хотелось бы иметь возможность проверить, произошло ли это событие или еще нет. Как правило, лучшее решение – это положить в какой-нибудь цикл условное выражение и ждать только тогда, когда событие не произошло.

Шаблон производитель – потребитель

Широко применяемым шаблоном программирования, основанном на взаимодействии потоков является шаблон «производитель – потребитель» (Producer – Consumer).

Один поток (производитель) генерирует данные или, например, получает их из сети, второй поток (потребитель) обрабатывает эти данные. Данные помещаются одним потоком в некотором контейнере (обычно в очереди) и извлекаются оттуда другим потоком.



Возникает вопрос, как потоку-потребителю отслеживать появление новых данных?

Первый способ - в цикле проверять наличие данных. Но при этом идет бесполезный расход процессорного времени.

Второй способ - в цикле проверять наличие данных и в конце каждой итерации при отсутствии данных засыпать на некоторое время. В этом случае теряется синхронность процесса производства-потребления.

Третий способ - синхронизировать задачи методами `wait/notify`.

Четвертый способ - использовать специальные классы синхронизированной очереди `SynchronousQueue`, в котором операции записи и чтения являются синхронизированными.

Рассмотрим синхронизацию методами `wait/notify`.

Упражнение 9. Реализация шаблона «производитель – потребитель» методами `wait/notify`. Очередь реализуется объектом двунаправленной очереди `ArrayDeque<E>`. Производитель генерирует строки и методом `putData()` помещает их в очередь. Производитель заканчивает работу записью строки `"exit"`. Потребитель извлекает строки из очереди методом `getData()` и выводит их в консоль. Потребитель заканчивает работу, когда встретит строку `"exit"`.

```
//-----------------------------------------------
// Объект хранения данных (накопитель) data,
// к которому обращаются
// поток производителя и поток потребителя
class TaskData {
    private final Deque<String> data =
           new ArrayDeque<String>();
    //  Метод чтения из очереди
    public String getData() {
       synchronized (data) {
          try {
             if (data.size() == 0)
              // ожидаем появление данных пол секунды
```

```
                data.wait(500);
            } catch (Exception e) {
            }
            return data.poll();
        }
    }
    //  Метод записи в очередь
    public void putData(String s) {
        synchronized (data) {
            data.add(s);
           // уведомляем ожидающий поток о появлении данных
            data.notify();
        }
    }
}
// ---------------------------------------------------------
// Класс производителя - порождает данные
class Producer implements Runnable {
    private final TaskData dest;
    private final int count;// кол-во производимых продуктов
    //  Конструктор класса
    public Producer(TaskData dest, int count) {
       this.dest = dest;
       this.count = count;
    }
    // Метод описывающий работу производителя
    @Override
    public void run() {
        try {
           for (int i = 0; i < count; ++i) {
              System.out.println("Producer put data - "+i);
              dest.putData("product - " + i);
              Thread.sleep(500); // засыпаем на пол-секунды
            }
         } catch (Exception e) {
         }
         dest.putData("exit");
         System.out.println("Producer thread finished");
    }
}
// ---------------------------------------------------------
// Класс потребителя - при появлении данных, выводим в консоль
// потребитель завершает работу, если встречает строку "exit"
class Consumer implements Runnable {
    final TaskData data;
```

```
    boolean cheking = true;
    //  Конструктор
    public Consumer(TaskData data) {
       this.data = data;
    }
    // Метод описывающий работу потребителя
    @Override
    public void run() {
        String s;
        while (cheking) {
          s = data.getData();
          if (s == null)
            System.out.println("Consumer timeout: " + cheking);
          else {
            System.out.println("Consumer get item: " + s);
            if (s.equals("exit"))
               cheking=false;
           }
        }
        System.out.println("Cosumer thread finished");
    }
}
//-----------------------------------
//  Класс тестирования
public class WaitNotifyDemo {
    public static void main(String[] args) {
        int count = 5;
        TaskData data = new TaskData();
        new Thread(new Producer(data, count)).start();
        new Thread(new Consumer(data)).start();
    }
}
```

Результат

```
Search  Debug  Shell  Servers  Console
<terminated> T07_WaitNotify [Java Application] E:\Program F
Producer put data - 0
Consumer get item: produced - 0
Producer put data - 1
Consumer timeout: true
Consumer get item: produced - 1
Consumer timeout: true
Producer put data - 2
Consumer get item: produced - 2
Consumer timeout: true
Producer put data - 3
Consumer get item: produced - 3
Consumer timeout: true
Producer put data - 4
Consumer get item: produced - 4
Producer thread finished
Consumer timeout: true
Consumer get item: exit
Cosumer thread finished
```

Рассмотрим реализацию шаблона «производитель-потребитель» с использованием синхронизированной очереди `SynchronousQueue`. Когда поток-производитель записывает данные в очередь `SynchronousQueue` методом `put()`, он блокируется до тех пор, пока другой поток не заберет этот элемент из нее методом `take()`. Соответственно, если поток пытается взять и удалить элемент из очереди, а элемента там нет, то этот поток блокируется до тех пор, пока другой поток не положит элемент в очередь. Объект SynchronousQueue так назван не безосновательно, он *передает данные синхронизированно* в другой поток; он ждет пока кто-то заберет данные, вместо того чтобы просто положить их и завершиться. Необходимость в методах `wait/notify` в данном случае отпадает.

Упражнение 10. Изменим пример упражнения 9. Для хранения данных будем использовать класс `SynchronousQueue`.

```
public class SynchroQueueDemo {
  public static void main(String[] args) {
     int count = 5;
     BlockingQueue<String> data = new ArrayBlockingQueue<>(10);
     new Thread(new Producer1(data, count)).start();
     new Thread(new Consumer1(data)).start();
  }
}
// ---------------------------------------------------------
//
class Producer1 implements Runnable {
     private BlockingQueue<String> data;
     private int count;
```

```
    public Producer1(BlockingQueue<String> data, int count) {
        this.data = data;
        this.count = count;
    }
    @Override
    public void run() {
        try {
          for (int i = 0; i < count; ++i) {
             System.out.println("Producer put data - " + i);
             String s = "produced - " + i;
             data.put(s);
             Thread.sleep(500); // засыпаем на 0,5сек
          }
          data.put("exit");
        } catch (Exception e) {
        }
        System.out.println("Producer thread finished");
    }
}
// ------------------------------------------------------------
class Consumer1 implements Runnable {
    private BlockingQueue<String> data;
    boolean cheking = true;

    public Consumer1(BlockingQueue<String> data) {
        this.data = data;
    }

    @Override
    public void run() {
      String s = null;
      try {
        while (cheking) {
          s = data.take();
          if (s == null)
           System.out.println("Consumer timeout:"+ cheking);
          else {
           System.out.println("Consumer get item: " + s);
           if (s.equals("exit"))
               cheking = false;
             }
          }
        } catch (InterruptedException e) {
             e.printStackTrace();
        }
```

```
            System.out.println("Cosumer thread finished");
        }
}
```

Результат будет таким же, как в предыдущем примере

2. Задание

Создать проект для многопоточного приложения по индивидуальным заданиям 1 и 2.

Примечание. Чтение из файла и запись в файл